MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 6
Operating Systems
Topic: Inside the xv6 File System

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 READING WHAT XV6 SHOWS **4**

2.1 Command Execution and Output 4

2.2 Analysis and Observations 4

3 THE MYSTAT PROGRAM AND HARD LINKS **6**

3.1 Implementation of `user/mystat.c` 6

3.2 Execution and Hard Link Trace 6

3.3 Analysis and Observations 7

4 MEASURING FILE SYSTEM LIMITS **8**

4.1 Inspection of Header Constants and Layout 8

4.2 Analysis and Observations 8

5 RECURSIVE DIRECTORY SEARCH (FIND) **9**

5.1 Implementation of `user/find.c` 9

5.2 Execution and Verification 10

5.3 Stretch Goal Evaluation 10

CONCLUSION **11**

REFERENCES **12**

1 INTRODUCTION AND GOAL

1.1 Introduction

The xv6 operating system provides a compact and fully readable implementation of a UNIX-like file system. Despite its minimal code base of only a few hundred lines, it encompasses foundational storage concepts including on-disk and in-memory inodes, directory tables, hard link referencing, disk logging via a journal, and buffer cache management. Investigating the internal disk layout and userland utilities of xv6 allows examining fundamental file system primitives without the configuration overhead of enterprise operating systems.

1.2 Lab Objective

The primary goals of this laboratory work are:

- inspecting the root directory structure and file typing mechanisms in xv6;
- implementing the custom userland program `mystat` to parse and inspect `struct stat`;
- tracing the life cycle of file references and data blocks during hard link operations and unlinking;
- evaluating the storage limits determined by architectural parameters in `kernel/fs.h`;
- implementing the recursive directory search program `find` and verifying its execution across nested directory hierarchies.

2 READING WHAT XV6 SHOWS

2.1 Command Execution and Output

The root filesystem and a newly created directory `a` were inspected using the native `ls` utility:

```
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 36728
echo            2 4 35592
forktest        2 5 18136
grep            2 6 44184
init            2 7 35976
kill            2 8 35512
ln              2 9 35304
ls              2 10 42880
mkdir           2 11 35568
rm              2 12 35552
sh              2 13 58312
stressfs        2 14 36416
usertests       2 15 206800
grind           2 16 51824
wc              2 17 37728
zombie          2 18 34848
logstress       2 19 37560
forphan         2 20 36352
dorphan         2 21 35800
sync            2 22 34944
sleep           2 23 35280
args            2 24 35272
console         3 25 0

$ mkdir a
$ ls a
.               1 26 32
..              1 1 1024
```

2.2 Analysis and Observations

a) **Meaning of types 1, 2, and 3:** Inspecting `kernel/stat.h` reveals the file type enumeration:

- `T_DIR (1)`: Directory entry.
- `T_FILE (2)`: Regular data file.
- `T_DEVICE (3)`: Special device file (e.g., the console).

b) **Root directory inode and loopback references:** The root directory has inode number **1** (defined by `ROOTINO` in `kernel/fs.h`). In the root directory, both `.` (current directory) and `..` (parent directory) refer to inode **1** because the root directory has no parent; it is the top-level anchor of the tree.

c) **Empty directory sizing and entries:** An empty directory `a` reports a size of **32 bytes**. Each directory entry in xv6 is represented by `struct dirent` (defined in `kernel/fs.h`), containing an unsigned

2-byte inode number (`ushort inum`) and a 14-byte name buffer (`char name[DIRSIZ]`), totaling exactly **16 bytes** per entry. Hence, $32/16 = 2$ entries, which correspond exactly to `.` and `..`.

3 THE MYSTAT PROGRAM AND HARD LINKS

3.1 Implementation of `user/mystat.c`

The `mystat` utility was created to invoke `stat()` and format the metadata fields:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *typename(int t)
{
  switch (t) {
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  default:       return "?";
  }
}

int main(int argc, char *argv[])
{
  struct stat st;

  if (argc < 2) {
    fprintf(2, "usage: mystat <file...>\n");
    exit(1);
  }

  for (int i = 1; i < argc; i++) {
    if (stat(argv[i], &st) < 0) {
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

3.2 Execution and Hard Link Trace

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, 1024 bytes
console: device, inode 27, links 1, size 0 bytes

$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes

$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes

$ cat r2 | wc
48 336 2441
```

3.3 Analysis and Observations

a) **Link count dynamics:** After running `ln README r2`, both `README` and `r2` point to inode **2**, and the link counter `nlink` for both increments from 1 to 2. The type, inode number, and file size remain completely identical. After executing `rm README`, the entry `README` is removed from the directory table, and the link counter on inode 2 decrements to 1. The data blocks remain entirely intact, allowing `cat r2 | wc` to read the full 2441 bytes.

b) **Unlinking and block deallocation in xv6:** In `kernel/sysfile.c`, the system call function `sys_unlink()` decrements the on-disk inode link count via `ip->nlink--` and commits the change with `iupdate(ip)`. The underlying data blocks are **not** freed immediately in `sys_unlink`. Instead, data blocks are deallocated inside `iput()` in `kernel/fs.c`: when an inode's in-memory reference count drops to 0 and `ip->nlink == 0`, `iput()` calls `itrunc(ip)`, which traverses all direct and indirect block pointers and marks them free in the block bitmap.

c) **Fields of `struct dinode`:** Examining `kernel/fs.h` confirms that xv6 inodes contain no access control bits, ownership UIDs/GIDs, or modification timestamps:

- `short type`: File type (`T_DIR`, `T_FILE`, `T_DEVICE`).
- `short major`, `short minor`: Major and minor device numbers.
- `short nlink`: Number of directory references pointing to this inode.
- `uint size`: Logical file size in bytes.
- `uint addrs[NDIRECT+1]`: Direct data block addresses plus one indirect block address.

4 MEASURING FILE SYSTEM LIMITS

4.1 Inspection of Header Constants and Layout

Executing the search in `kernel/fs.h` and capturing disk generation yields:

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14

$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

4.2 Analysis and Observations

a) **Long directory entry truncation:** Running `mkdir averyveryverylongname` followed by `ls` displays the directory name as `averyveryveryl`. This behavior is governed by the constant `DIRSIZ` defined as **14** in `kernel/fs.h`. Any character beyond 14 bytes is truncated by `dirlink()` when writing into `struct dirent`.

b) **Maximum file size calculation:** The maximum file capacity supported by the xv6 filesystem is calculated from the direct and indirect block address layout:

$$\text{Block Size } (BSIZE) = 1024 \text{ bytes} \tag{4.1}$$

$$\text{Block Pointer Size} = \text{sizeof}(\texttt{uint}) = 4 \text{ bytes} \tag{4.2}$$

$$\text{Number of Indirect Blocks } (NINDIRECT) = \frac{1024}{4} = 256 \text{ blocks} \tag{4.3}$$

$$\text{Total Addressable Blocks} = NDIRECT + NINDIRECT = 12 + 256 = 268 \text{ blocks} \tag{4.4}$$

$$\text{Maximum File Size} = 268 \times 1024 = 274,432 \text{ bytes } (\approx 268 \text{ KB}) \tag{4.5}$$

c) **Disk block budget:** According to the observed `nmeta` output, the entire virtual disk consists of **2000 blocks**:

- Metadata blocks ($nmeta = 47$): 1 boot block, 1 superblock, 31 log/journal blocks, 13 inode blocks, and 1 bitmap block.
- Data blocks: $2000 - 47 =$ **1953** blocks available for file and directory content.

5 RECURSIVE DIRECTORY SEARCH (FIND)

5.1 Implementation of user/find.c

The find program traverses directory trees recursively, reading struct dirent entries, checking against inum == 0, avoiding . and .., and supporting the optional -n stretch flag:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void find(char *path, char *target, int show_ino)
{
  char buf[512];
  char *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if (fstat(fd, &st) < 0 || st.type != T_DIR) {
    close(fd);
    return;
  }
  if (strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while (read(fd, &de, sizeof(de)) == sizeof(de)) {
    if (de.inum == 0) continue;
    if (strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0) continue;

    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;

    if (stat(buf, &st) < 0) continue;

    if (strcmp(de.name, target) == 0) {
      if (show_ino)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }

    if (st.type == T_DIR)
      find(buf, target, show_ino);
  }
  close(fd);
}

int main(int argc, char *argv[])
```

```
{
  int show_ino = 0;
  if (argc < 3) {
    fprintf(2, "usage: find <path> <name> [-n]\n");
    exit(1);
  }
  if (argc >= 4 && strcmp(argv[3], "-n") == 0)
    show_ino = 1;
  find(argv[1], argv[2], show_ino);
  exit(0);
}
```

5.2 Execution and Verification

The test tree was built on the disk image and queried:

```
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ find . r2
./r2
./a/b/r2
./a/r2

$ find a b
a/b

$ find . r2 -n
./r2 2
./a/b/r2 31
./a/r2 32
```

5.3 Stretch Goal Evaluation

Inspecting the output of `find . r2 -n` demonstrates that `./r2` has inode number **2**, which is the hard link created to the `README` file (inode 2). Conversely, `./a/r2` (inode 32) and `./a/b/r2` (inode 31) are newly created distinct files allocated with distinct inode numbers. Thus, the stretch flag verifies that only `./r2` shares the exact underlying inode and data blocks of the original file, while the others are independent filesystem nodes that simply share the same basename.

CONCLUSION

This laboratory work provided a practical, code-level exploration of the xv6 file system architecture. By inspecting data structures in `kernel/fs.h` and implementing userland system utilities, the internal lifecycle of files and directories was clearly demonstrated.

Key insights gained from this laboratory include:

- **Inode-Directory decoupling:** File names exist strictly as 16-byte records in directory tables mapped to inode indices, whereas data blocks, types, and link counters reside inside the inode structure.
- **Reference counting and deferred freeing:** Creating a hard link simply increments `nlink`. Data block reclaiming is deferred to `iput()` when both the directory link count and active file descriptor count drop to zero.
- **Architectural limits:** Fixed constants such as `DIRSIZ` and `BSIZE` dictate strict limits, including the maximum directory entry name length (14 characters) and maximum addressable file capacity (268 KB).
- **Recursive traversal:** Implementing `find` illustrated directory file parsing, descriptor leak prevention, and infinite recursion mitigation by bypassing `.` and `..`.

Overall, this practical work deepened my comprehension of operating system storage hierarchies and reinforced how filesystem system calls translate into concrete disk block operations.

REFERENCES

[1] Russ Cox, Frans Kaashoek, Robert Morris, *xv6: a simple, Unix-like teaching operating system*, MIT CSAIL, 2020.

[2] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, 2018.

[3] MIT 6.1810 / 6.828 Operating System Engineering, *Lab util: Unix utilities (find)*, MIT OpenCourseWare.